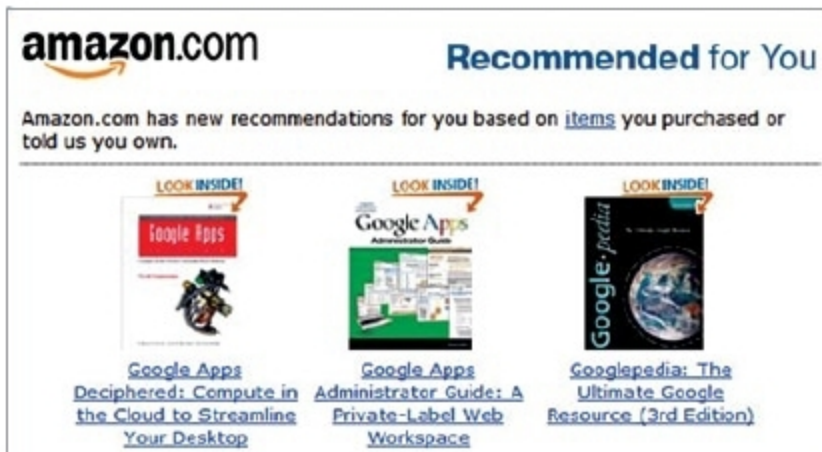




Amazon's recommendations for customers

system looks for users who have made the same purchases as you and rated these items similarly. The system then removes already-purchased items from the list of other items these similar customers purchased and rated highly. Rest of the items are then recommended to you. Two popular versions of these algorithms are collaborative filtering and cluster models.

Collaborative filtering. A collaborative filtering algorithm represents a customer as an n -dimensional vector (n being the number of item types that can be considered distinct). Vector components are positive for positively-rated items and negative for negatively-rated ones.

To give lesser weight to already popular items, the algorithm multiplies the inverse of the number of customers who have rated/purchased an item with vector components. The algorithm then finds similar customers using cosine similarity. Cosine similarity is a measure of similarity between two non-zero vectors of an inner product space that measures the cosine of the angle between them. Hence, in an n -dimensional vector, cosine of the angle between vectors represents similarity.

The algorithm computes $\cos\theta$ as dot product of vectors or product of their magnitudes. When $\cos\theta$ approaches 1, vectors are closer in direction, the more similar the contents are. And when $\cos\theta$ approaches -1, contents are distinct.

The algorithm gives recommendations based on similarity between customers. Amazon uses collaborative filtering, but similarity is not found between users but between items. So, the system looks for items that are bought together with the item

that you have chosen. Amazon maintains a similar items table that it generates offline for this purpose.

Cluster models. Cluster models divide users into pre-defined segments and use classification (of users to these segments) for recommendations. Classification is a specialised

statistics field, wherein on the basis of certain rules items are classified. Rules can be pre-defined or developed using machine learning. Spam filters are the most common example of classification.

For recommendation systems, the system begins with certain segments by using either clustering algorithm or assisted learning with machine learning—examples of different customers that belong to a segment are used as an example for machine learning algorithms to learn from and then do future classification based on the understanding.

The latter technique uses pre-defined segments. Newer segments are added/merged on the basis of density of each segment in terms of customers added. The idea is to have not too many segments (leading to very few customers in each) or too few segments (wherein segments in itself are general/vague).

Cluster models group numerous customers together in a segment, match a user to the segment and then consider all customers in that segment—similar customers for the purpose of making recommendations.

Search-based methods. This is an algorithm that is not really an algorithm. It solves the problem of recommendations through search for related items. IMDB most likely uses this model (although, it tends to consider user ratings, too. Most likely, it uses a mix of collaborative filtering and search-based methods.).

As per this model, if I (as a registered member) highly rate a movie like *The Godfather*, the system would suggest movies with similar actors (Marlon Brando as the lead) or genres (crime/gangster). The system makes

a search query on inverted indexes where most-likely tags (of genre) are associated with documents.

This is a much more precise form of free-text search, similar to what search engines use. This method suffers from the flaw that, on a number of occasions, suggestions are too broad (on the basis of genre or popularity).

Possibilities with current recommendation systems

Most recommendation systems are built after years of research, and continuation of the algorithm is a proof of their working. However, there are issues pertaining to these systems that need to be addressed.

One major problem is lack of communication between different systems. By this I mean that most customers have distinct accounts on various platforms. These tend to depict n -dimensional depth of a user on various platforms.

For example, I enjoy watching movies and reading books. I have accounts on IMDB and Goodreads, respectively, for recommendations. These, however, are not synced with my recommendations on Amazon. So ideally, if I add a book to my To Read list on Goodreads, it should reflect as recommendations on Amazon, since Amazon owns Goodreads, too.

Similarly, if I read a book that I did not buy from Amazon, my Read list on Goodreads should be a hint to Amazon and should be removed from suggestions. And a movie that I added to my Watchlist on IMDB should show up as a suggestion on my Amazon Prime Account, and so on.

Websites that list things and classify these into various tags can be of great use in terms of suggestions. Complex items like books and movies tend to suffer from the problem of classification. While most movies/books fall under multiple genres/sub-genres, their classification and finding similar items in such categories is often problematic.

The problem arises because most similar items are loosely coupled in terms of their similarity. For example, on IMDB, people who are recommended *The Lord of the Rings* are also recommended *Pulp Fiction*. For any